

Übungsblatt 6: Multicore-Echtzeit und Shared Memory

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

In dieser Übung konfigurieren Sie ein asymmetrisches Multicore-System (**AMP**) auf dem **STM32H755**. Der Cortex-M7 fungiert als Bare-Metal-Master für hohen Durchsatz, während der Cortex-M4 als Echtzeit-Kern **FreeRTOS** ausführt. Sie etablieren einen Ping-Pong-Datenaustausch über einen Shared-Memory-Bereich im **SRAM4** und provozieren sowie beheben den klassischen *Cache Coherency Failure* mithilfe der **Memory Protection Unit (MPU)** auf Hardware-Ebene.

Aufgabe 1: Dual-Core-Setup und Boot-Prozess

Im Gegensatz zu klassischen SMP-Systemen starten beim STM32H755 nicht beide Kerne gleichzeitig. Der Cortex-M7 bootet als Master, während der Cortex-M4 im Reset-Zustand verharrt.

1. Starten Sie ein neues Projekt für das Board **STM32H755ZI-Q** in der STM32CubeIDE. Aktivieren Sie beide Prozessorkerne.
2. Aktivieren Sie FreeRTOS ausschließlich für den Cortex-M4 (*Asymmetric OS Design*).
3. Ergänzen Sie den generierten C-Code so, dass der Cortex-M7 den Cortex-M4 beim Systemstart aufweckt.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

In der `main.c` des **CM7-Projekts** muss unmittelbar nach der HAL- und SystemClock-Initialisierung der Cortex-M4 explizit aus dem Hardware-Reset befreit werden:

```

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    /* WICHTIG: M4 aus dem Hardware-Reset befreien */
    HAL_PWREx_ReleaseCore(PWR_CORE_CPU2);

    /* ... restlicher Code des M7 ... */
}

```

Ohne den Aufruf von `HAL_PWREx_ReleaseCore(PWR_CORE_CPU2)` bleibt der M4 im Reset; FreeRTOS auf dem M4 startet niemals.

Aufgabe 2: Shared Memory und Linker-Script

Die Kerne benötigen einen gemeinsamen Speicherbereich. Nutzen Sie den **SRAM4** in der D3-Domäne ab der Adresse `0x38000000` (64KB), da dieser für beide Kerne gut erreichbar ist.

1. Legen Sie eine Header-Datei `shared_data.h` im Ordner `Common/Inc` an, die ein C-Struct mit den Flags `m7_ready`, `m4_ready` und einem `payload_value` enthält.
2. Deklarieren Sie die Shared-Variable mit dem Schlüsselwort `volatile` und weisen Sie sie per GCC-Attribut der Sektion `.shared_memory` zu. Legen Sie die Definition (ohne `extern`) in genau einer `.c`-Datei im Common-Ordner an und binden Sie den Header in beiden Projekten ein.
3. Öffnen Sie die Linker-Skripte (`.ld`) **beider** Kerne und fügen Sie den Speicherbereich (RAM_D3 ab `0x38000000`, 64K) sowie die Sektion `.shared_memory` hinzu. Achten Sie zwingend auf das Flag (NOLOAD)!

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

1. Header-Datei (`shared_data.h`):

```

#ifndef SHARED_DATA_H
#define SHARED_DATA_H

#include <stdint.h>

typedef struct {
    uint8_t m7_ready;
    uint8_t m4_ready;
    uint32_t payload_value;
} SharedData_t;

```

```

/* Deklaration: Sektions-Zuweisung + volatile */
extern __attribute__((section(".shared_memory")))
    volatile SharedData_t sd;

#endif

```

Definition (z. B. in Common/Src/shared_data.c):

```

#include "shared_data.h"

__attribute__((section(".shared_memory")))
volatile SharedData_t sd;

```

`volatile` zwingt den C-Compiler, bei jedem Zugriff frisch aus dem Speicher zu lesen bzw. zu schreiben und Werte nicht in CPU-Registern wegzuo optimieren. Es löst **kein** Cache-Kohärenz-Problem auf Hardware-Ebene—das ist Aufgabe der MPU (siehe Aufgabe 4).

2. Anpassung der Linker-Skripte (STM32H755..._FLASH.ld in CM7 und CM4):

Im MEMORY-Block:

```

RAM_D3 (xrw) : ORIGIN = 0x38000000, LENGTH = 64K

```

Im SECTIONS-Block:

```

.shared_memory (NOLOAD) :
{
    . = ALIGN(4);
    *(.shared_memory)
} > RAM_D3

```

Warum (NOLOAD)?

Ohne `NOLOAD` behandelt der Startup-Code die Sektion wie `.bss` und nullt sie beim Boot. Der M4 startet typischerweise kurz nach dem M7. Hat der M7 bereits Flags oder Payload in den SRAM4 geschrieben, würde der M4-Startup diesen Shared Memory erneut mit Nullen überschreiben und die Initialdaten vernichten. `NOLOAD` verbietet genau dieses Nullen: der Bereich wird reserviert, aber nicht initialisiert.

Aufgabe 3: Ping-Pong-Logik (Inter-Core-Kommunikation)

Implementieren Sie ein asymmetrisches Handshake-Protokoll zwischen den Kernen.

1. **Cortex-M7 (Bare-Metal):** Initialisieren Sie die Werte (`payload_value = 0`, `m7_ready = 1`). Warten Sie in der Super-Loop per Polling darauf, dass der M4 geantwortet hat (`m4_ready == 1`). Geben Sie den Wert per `printf()` aus, quittieren Sie das Flag, erhöhen Sie den Wert, warten Sie 500 ms zur Lesbarkeit und übergeben Sie wieder an den M4 (`m7_ready = 1`).
2. **Cortex-M4 (FreeRTOS):** Die RTOS-Task soll auf Daten des M7 warten, solange `m7_ready == 0`. Nutzen Sie dabei `osDelay(1)`, damit die Task die CPU nicht per Busy-Wait blockiert. Sobald Daten anliegen, quittieren Sie das M7-Flag, inkrementieren Sie den Payload und signalisieren Sie die Antwort (`m4_ready = 1`).

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Logik Cortex-M7 (Bare-Metal in main.c):

```
/* Initiale Übergabe */
sd.payload_value = 0;
sd.m7_ready = 1;

while (1) {
    /* Polling ist im Bare-Metal als Demo akzeptabel */
    if (sd.m4_ready == 1) {
        printf("M4 hat geantwortet: %lu\n",
               (unsigned long)sd.payload_value);
        sd.m4_ready = 0;          /* Quittieren */
        sd.payload_value += 1;
        HAL_Delay(500);           /* Demo: Lesbarkeit am Terminal */
        sd.m7_ready = 1;         /* Ball zurück zum M4 */
    }
}
```

Logik Cortex-M4 (FreeRTOS-Task, z. B. in freertos.c / StartDefaultTask):

```
void StartDefaultTask(void *argument)
{
    for (;;) {
        /* Warten, bis M7 Daten bereitstellt */
        while (sd.m7_ready == 0) {
            osDelay(1); /* Task → Blocked; CPU nicht busy-warten */
        }

        sd.m7_ready = 0;          /* Quittieren */
        sd.payload_value += 1;

        sd.m4_ready = 1;         /* Antwort an M7 signalisieren */
    }
}
```

Warum osDelay(1) auf dem M4, aber HAL_Delay(500) auf dem M7?

- Auf dem **M4** läuft FreeRTOS. Ein Busy-Wait (`while (sd.m7_ready == 0) { }` ohne Delay) würde die Task dauerhaft im Zustand *Running* halten und andere Ready-Tasks verhungern lassen. `osDelay(1)` legt die Task kurz in den Zustand *Blocked*; der Scheduler kann andere Tasks bedienen.
- Auf dem **M7** gibt es kein OS. `HAL_Delay(500)` ist hier eine bewusste Demo-Pause für die Terminal-Lesbarkeit (Busy-Wait im Bare-Metal). In einem produktiven M7-Design würde man statt Endlos-Polling Interrupts, DMA oder Hardware-Semaphoren (HSEM, Vorlesung 8) nutzen.

Aufgabe 4: Cache-Kohärenz und System-Freeze

Wenn Sie den Code aus Aufgabe 3 flashen, friert das System nach der initialen Ausgabe ein: Die Konsole bleibt stumm, der Cortex-M4 verharrt in seiner Warteschleife.

1. Erklären Sie physikalisch, warum dieser *Cache Coherency Failure* auftritt.
2. Lösen Sie das Problem hardwareseitig über die Memory Protection Unit (MPU) des Cortex-M7 im CubeMX-Configurator. Geben Sie die genauen Konfigurationsparameter an.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

1. Erklärung des Cache Coherency Failure

Der Cortex-M7 besitzt einen L1-Data-Cache, der standardmäßig im Verfahren **Write-Back** arbeitet:

1. Der M7 führt `sd.m7_ready = 1` aus.
2. Die Hardware schreibt die 1 zunächst **nur** in den internen L1-Cache des M7.
3. Der physikalische **SRAM4** behält den veralteten Wert (0), bis die Cache-Zeile später verdrängt (evicted) und zurückgeschrieben wird.
4. Der Cortex-M4 hat **keinen** Data-Cache und liest über die Bus-Matrix direkt den SRAM4—dort steht weiterhin 0.
5. Der M4 wartet daher endlos; Cache und Hauptspeicher sind nicht mehr kohärent.

Hardware-Snooping (wie bei Desktop-CPUs) existiert beim STM32H755 aus Latenz- und Determinismusgründen nicht. `volatile` hilft hier nicht: Es betrifft nur Compiler-Optimierungen, nicht den L1-Cache.

2. Lösung über die MPU (Non-Cacheable)

Statt den Cache fehleranfällig per Software zu reinigen (`SCB_CleanDCache`), entziehen wir dem Cache die Erlaubnis, den SRAM4 zu puffern.

Im CubeMX: *System Core* → *CORTEX_M7* → Reiter *MPU Control*, Region aktivieren:

Parameter	Wert
Base Address	0x38000000
Size	64 KB
Access Permission	ALL ACCESS
TEX / Shareable / Cacheable	Disable Cache (<i>Non-Cacheable</i>)

Wichtig: Die Region als **Non-Cacheable** konfigurieren—nicht als *Strongly Ordered*. Non-Cacheable reicht aus, damit Schreibzugriffe des M7 den L1-Cache umgehen und direkt über die Bus-Matrix in den SRAM4 gelangen; der M4 sieht die neuen Daten sofort. Code neu generieren und flashen.

Kontrollversuch: Kommentieren Sie `HAL_MPU_Enable(...)` aus und flashen Sie erneut—das System friert wieder ein. Damit ist der Architekturfehler reproduzierbar bewiesen.